

Student Name: Rana Omarhan

Student ID:230103191

Session / Date: 02-N 08.10.26

Assigned GPU ID: 0 (Tesla T4)

SECTION 1

NVIDIA Driver Version: 580.82.07

CUDA Version Supported: 13.0

Allocated GPU Name: Tesla T4

Total Available VRAM: 15360 MiB

Compute Capability Major.Minor: 7.5

Max Threads Per Block: 1024

SECTION 2

Array Size (N)	CPU Time (ms)	GPU Total (ms)	GPU Kernel Only (ms)	Winner (CPU vs GPU)
100	0.0560	1.4407	0.0852	CPU
1,000	0.2837	0.5651	0.0637	CPU
10,000	2.5870	0.5537	0.0597	GPU
100,000	25.9112	0.6639	0.0655	GPU
1,000,000	281.5825	3.0983	0.1304	GPU
5,000,000	N/A (skip)	13.8746	0.2720	GPU

Array Size (N)	CPU Time (ms)	GPU Total (ms)	GPU Kernel Only (ms)	Winner (CPU vs GPU)
10,000,000	N/A (skip)	25.4073	0.6092	GPU

Q1. What is the empirical Crossover Point N^* on your machine? (i.e., at what array size did GPU Total Time finally become faster than CPU Time?):

The crossover point N^* is between **1,000 and 10,000**. At $N=1,000$, CPU time (0.2837 ms) is still faster than GPU Total time (0.5651 ms). By $N=10,000$, the GPU becomes significantly faster (0.5537 ms vs CPU's 2.5870 ms). Therefore, N^* is roughly around 2,000 to 5,000.

Q2. At $N = 100$, why is GPU Total Time dramatically slower than CPU Time? Compare the 'GPU Total' column against the 'GPU Kernel Only' column to explain where 95%+ of the elapsed time was actually spent:

At $N=100$, GPU Total Time is 1.4407 ms, while GPU Kernel Only time is just 0.0852 ms. This means over 94% of the time was spent on **PCIe data transfer overhead** (Host-to-Device and Device-to-Host copying), not on computation. The fixed latency of moving data across the PCIe bus dominates when the array is very small. CPU time (0.0560 ms) is faster because it avoids this transfer cost entirely.

SECTION 3

Observed behavior / Console output from defective kernel:

Execution finished without crashing. Elements processed: 1000. Last element: 999.0.

The 24 extra threads (indices 1000–1023) wrote out-of-bounds, but because CUDA memory allocations are padded to alignment boundaries, the writes landed in unused padding memory instead of triggering a segfault. This demonstrates that out-of-bounds GPU writes can silently corrupt adjacent memory without throwing a Python IndexError.

MANDATORY CALCULATION: For $N = 37$ and Block Size = 8:

Total Blocks Allocated: 5

Total Hardware Threads Launched: 40

Number of Active Working Threads: 37

Number of Idle Guarded Threads: 3

SECTION 4

--- Running Naive Kernel (5 Trials) ---

Trial 1: Naive Sum = 2.0 | Expected = 50000.0 | Lost Updates = 49998.0

Trial 2: Naive Sum = 2.0 | Expected = 50000.0 | Lost Updates = 49998.0

Trial 3: Naive Sum = 2.0 | Expected = 50000.0 | Lost Updates = 49998.0

Trial 4: Naive Sum = 2.0 | Expected = 50000.0 | Lost Updates = 49998.0

Trial 5: Naive Sum = 2.0 | Expected = 50000.0 | Lost Updates = 49998.0

--- Running Atomic Kernel ---

Atomic Sum = 50000.0 | Expected = 50000.0 | Time = 0.6681 ms

Naive Time = 0.4988 ms

Explain why atomic operations incur a massive execution penalty:

Atomic operations serialize updates to the same memory address. All 50,000 threads must queue up and update `d_total[0]` one at a time. The hardware lock introduces memory fence and serialization overhead, so the atomic kernel is much slower than the naive kernel. However, the naive kernel is only "fast" because it is doing millions of conflicting updates concurrently, resulting in massive data loss. Correctness requires the atomic penalty.

SECTION 5

Grid Dimensions:

Blocks in X: 64

Blocks in Y: 64

Total Blocks: 4096

Verified Pixel Value at Center Point (512, 512): 1.000

Verified Pixel Value at Outer Corner Point (0, 0): 0.000

SECTION 6

=====

OFFICIAL VERIFICATION CHECKSUM TOKEN: 40DE3885EC1A615B

=====

40DE3885EC1A615B